Computer Organization and Architecture

Module 2 –Addressing modes

Q. Define addressing mode. Explain any four basic addressing modes with syntax and examples.

ADDRESSING MODES

• The different ways in which the location of an operand is specified in an instruction are referred to as **Addressing Modes** (Table 2.1).

Table 2.1 Generic addressing modes

Name	Assembler syntax	Addressing function
Immediate	#Value	Operand = Value
Register	R*i*	EA = R*i*
Absolute (Direct)	LOC	EA = LOC
Indirect	(R*i*) (LOC)	EA = [R*i*] EA = [LOC]
Index	X(R*i*)	EA = [R*i*] + X
Base with index	(R*i*,R*j*)	EA = [R*i*] + [R*j*]
Base with index and offset	X(R*i*,R*j*)	EA = [R*i*] + [R*j*] + X
Relative	X(PC)	EA = [PC] + X
Autoincrement	(R*i*)+	EA = [R*i*]; Increment R*i*
Autodecrement	–(R*i*)	Decrement R*i*; EA = [R*i*]

EA = effective address
Value = a signed number

Different addressing modes are as follows:

1) Register Mode

• The operand is the contents of a register.
• The name (or address) of the register is given in the instruction.
• Registers are used as temporary storage locations where the data in a register are accessed.
• For example, the instruction

Move R1, R2 ;Copy content of register R1 into register R2.

2) Absolute (Direct) Mode

• The operand is in a memory-location.
• The address of memory-location is given explicitly in the instruction.
• The absolute mode can represent global variables in the program.
• For example, the instruction

Move LOC, R2 ;Copy content of memory-location LOC into register R2.

3) Immediate Mode

• The operand is given explicitly in the instruction.
• For example, the instruction

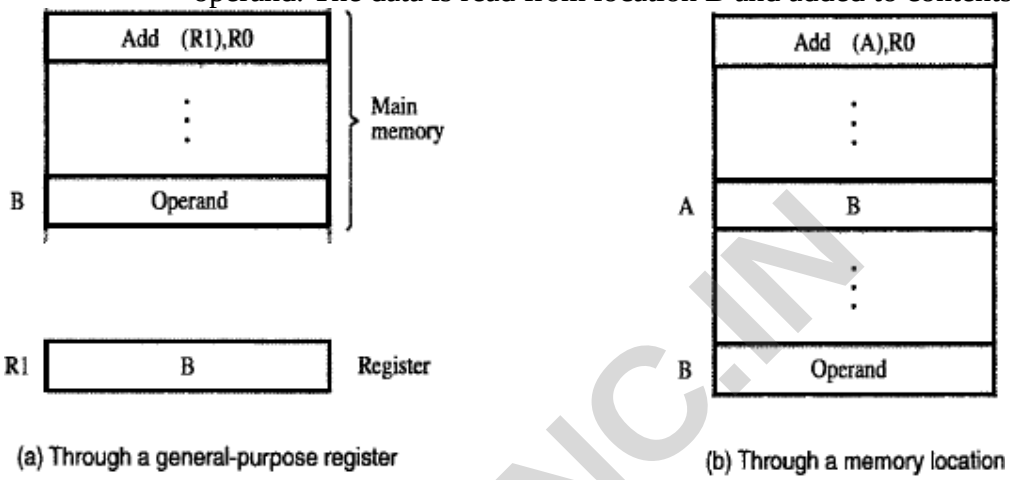
Move #200, R0 ;Place the value 200 in register R0.

• Clearly, the immediate mode is only used to specify the value of a source-operand.

4) Indirect Mode

The instruction provides information from which the new address of the operand can be determined. This address is called **Effective Address (EA)** of the operand.

• The EA of the operand is the contents of a register(or memory-location).
• The register (or memory-location) that contains the address of an operand is called a **Pointer**.
• We denote the indirection by
→ name of the register or
→ new address given in the instruction.
E.g: *Add (R1),R0* ;The operand is in memory. Register R1 gives the effective-address (B) of the operand. The data is read from location B and added to contents of register R0.



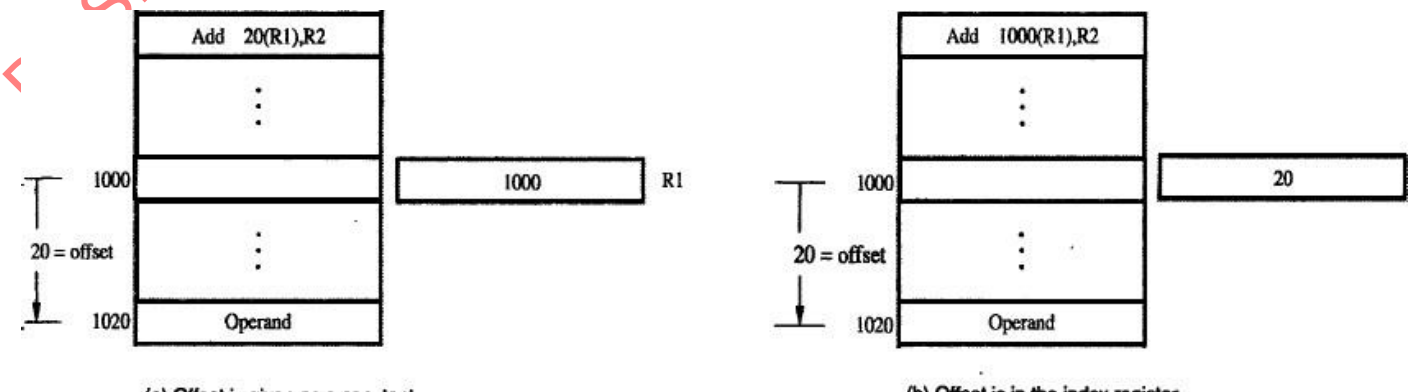
(a) Through a general-purpose register (b) Through a memory location

Figure 2.11 Indirect addressing.

• To execute the Add instruction in fig 2.11 (a), the processor uses the value which is in register R1, as the EA of the operand.
• It requests a read operation from the memory to read the contents of location B. The value read is the desired operand, which the processor adds to the contents of register R0.
• Indirect addressing through a memory-location is also possible as shown in fig 2.11(b). In this case, the processor first reads the contents of memory-location A, then requests a second read operation using the value B as an address to obtain the operand.

5) Index mode

• The operation is indicated as X(Ri)
where X=the constant value which defines an offset(also called a displacement).
Ri=the name of the index register which contains address of a new location.
• The effective-address of the operand is given by EA=X+[Ri]
• The contents of the index-register are not changed in the process of generating the effective- address.
• The constant X may be given either
→ as an explicit number or
→ as a symbolic-name representing a numerical value.



(a) Offset is given as a constant (b) Offset is in the index register

Figure 2.13 Indexed addressing.

• Fig(a) illustrates two ways of using the Index mode. In fig(a), the index register, R1, contains the address of a memory-location, and the value X defines an offset(also called a displacement) from this address to the location where the operand is found.
• To find EA of operand:
Eg: Add 20(R1), R2
EA=>1000+20=1020
• An alternative use is illustrated in fig(b). Here, the constant X corresponds to a memory address, and the contents of the index register define the offset to the operand. In either case, the effective-address is the sum of two values; one is given explicitly in the instruction, and the other is stored in a register.

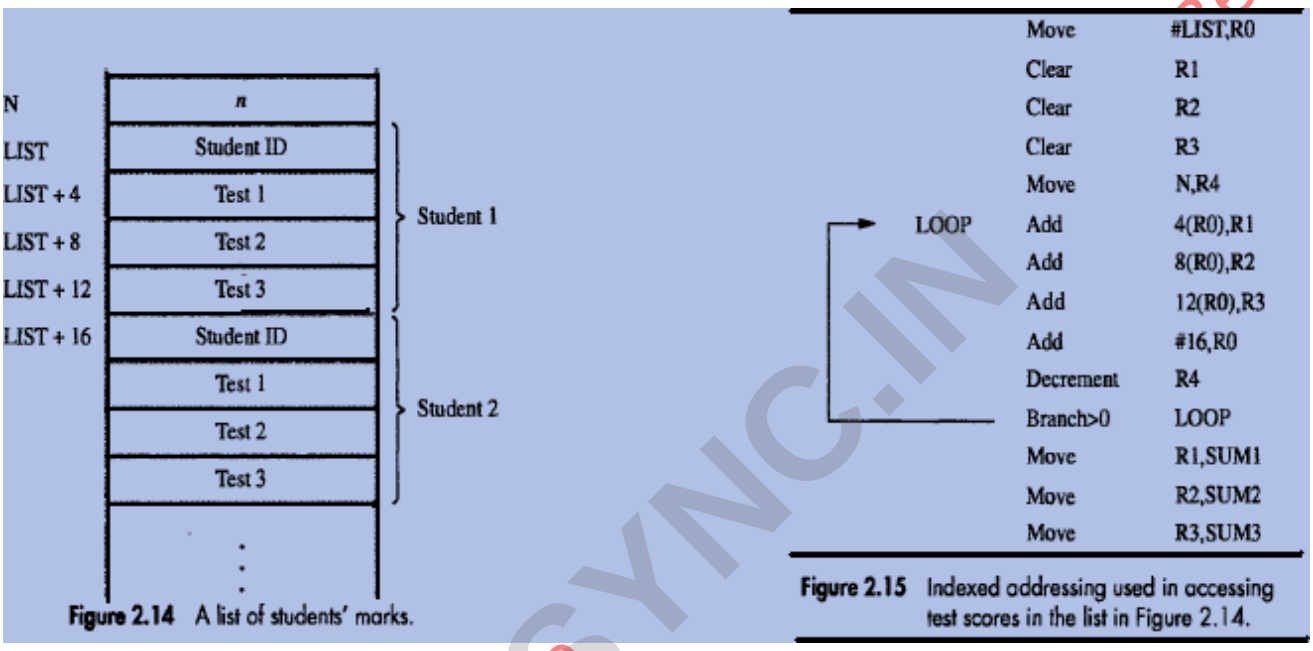


Figure 2.14 A list of students' marks.
Figure 2.15 Indexed addressing used in accessing test scores in the list in Figure 2.14.

6) Base with Index Mode
• Another version of the Index mode uses 2 registers which can be denoted as (Ri, Rj)
• Here, a second register may be used to contain the offset X.
• The second register is usually called the *base register*.
• The effective-address of the operand is given by EA=[Ri]+[Rj]
• This form of indexed addressing provides more flexibility in accessing operands because both components of the effective-address can be changed.

7) Base with Index & Offset Mode
• Another version of the Index mode uses 2 registers plus a constant, which can be denoted as X(Ri, Rj)
• The effective-address of the operand is given by EA=X+[Ri]+[Rj]
• This added flexibility is useful in accessing multiple components inside each item in a record, where the beginning of an item is specified by the (Ri, Rj) part of the addressing-mode. In other words, this mode implements a 3-dimensional array.

8) RELATIVE MODE
• This is similar to index-mode with one difference:
The effective-address is determined using the PC in place of the general purpose register Ri.
• The operation is indicated as X(PC).
• X(PC) denotes an effective-address of the operand which is X locations above or below the current

contents of PC.

- Since the addressed-location is identified "relative" to the PC, the name Relative mode is associated with this type of addressing.
- This mode is used commonly in conditional branch instructions.
- An instruction such as

Branch > 0 LOOP ;Causes program execution to go to the branch target location identified by name LOOP if branch condition is satisfied.

Additional Addressing Modes

9) Auto Increment Mode

- Effective-address of operand is contents of a register specified in the instruction (Fig: 2.16).
- After accessing the operand, the contents of this register are automatically incremented to point to the next item in a list.
- Implicitly, the increment amount is 1.
- This mode is denoted as

(Ri)+ ; where Ri=pointer-register.

10) Auto Decrement Mode

- The contents of a register specified in the instruction are first automatically decremented and are then used as the effective-address of the operand.
- This mode is denoted as

-(Ri) ;where Ri=pointer-register.

- These 2 modes can be used together to implement an important data structure called a stack.

Q. Write a program to add N numbers using indirect addressing mode.

Address	Contents		
	Move	N,R1	Initialization
	Move	#NUM1,R2	
	Clear	R0	
LOOP	Add	(R2),R0	
	Add	#4,R2	
	Decrement	R1	
	Branch>0	LOOP	
	Move	R0,SUM	

Figure 2.12 Use of indirect addressing in the program of Figure 2.10.

Program Explanation

- In above program, Register R2 is used as a pointer to the numbers in the list, and the operands are accessed indirectly through R2.
- The initialization-section of the program loads the counter-value n from memory-location N into R1 and uses the immediate addressing-mode to place the address value NUM1, which is the address of the first number in the list, into R2. Then it clears R0 to 0.
- The first two instructions in the loop implement the unspecified instruction block starting at LOOP.
- The first time through the loop, the instruction Add (R2), R0 fetches the operand at location NUM1 and adds it to R0.
- The second Add instruction adds 4 to the contents of the pointer R2, so that it will contain the address value NUM2 when the above instruction is executed in the second pass through the loop.

Q. Write a program to add N numbers using indirect and autoincrement addressing mode.

```
                Move        N,R1        }
                Move        #NUM1,R2    } Initialization
                Clear       R0          }
LOOP            Add         (R2)+,R0
                Decrement   R1
                Branch>0    LOOP
                Move        R0,SUM
```

Figure 2.16 The Autoincrement addressing mode used in the program of Figure 2.12.

Q. Compare Memory Mapped I/O with I/O mapped I/O.

Memory Mapped I/O	I/O Mapped I/O
1. it is a method to perform input/output Operations between CPU and peripheral Devices in a computer that uses one Address space for memory and I/O devices.	1. it is a method to perform input/output Operations between CPU and peripheral Devices in a computer that uses two Address space for memory and I/O devices.
2. Uses the same address space for both Memory and input-output devices.	2. Uses two separate address spaces for Memory and input-output devices.
3. The available address space for memory is less	3. The available address space for memory is more.
4. Uses the same instruction for both Memory and input-output operations.	4. Uses separate instructions for Memory and input-output operations.
5. Less efficient	5. More efficient

Q. Compare Source program with Object Program

Sl No.	Source Program	Object Program
1	A collection of computer instructions written using a human readable programming language.	A sequence of statements in binary that is generated after compiling the source program.
2	Contains English words according to the syntax of programming language.	Contains Binaries
3	Source code is another name for source program	Object code is another name for object program
4	Programmer can read the source program	Machine can read the object program
5	Programmer creates the source program	Compiler creates the object program
6	Source program is the input to the compiler	Object program is the output of the compiler.

Q. What are assembler directives? Write a program for addition of N numbers using assembler directives.

ASSEMBLER DIRECTIVES

- **Directives** are the assembler commands to the assembler concerning the program being assembled.
- These commands are not translated into machine opcode in the object-program.
- **EQU** informs the assembler about the value of an identifier (Figure: 2.18).
  Ex*: SUM EQU 200* ;Informs assembler that the name SUM should be replaced by the value 200.
- **ORIGIN** tells the assembler about the starting-address of memory-area to place the data block.
  Ex*:* ORIGIN 204 ;Instructs assembler to initiate data-block at memory-locations starting from 204.
- **DATAWORD** directive tells the assembler to load a value into the location.
  Ex: *N DATAWORD 100 ;*Informs the assembler to load data 100 into the memory-location N(204).
- **RESERVE** directive is used to reserve a block of memory.
  Ex: *NUM1 RESERVE 400 ;*declares a memory-block of 400 bytes is to be reserved for data.
- **END** directive tells the assembler that this is the end of the source-program text.
- **RETURN** directive identifies the point at which execution of the program should be terminated.
- Any statement that makes instructions or data being placed in a memory-location may be given a **label**. The label(say N or NUM1) is assigned a value equal to the address of that location.

	Memory address label	Operation	Addressing or data information
Assembler directives	SUM	EQU	200
		ORIGIN	204
	N	DATAWORD	100
	NUM1	RESERVE	400
		ORIGIN	100
Statements that	START	MOVE	N,R1
generate		MOVE	#NUM1,R2
machine		CLR	R0
instructions	LOOP	ADD	(R2),R0
		ADD	#4,R2
		DEC	R1
		BGTZ	LOOP
		MOVE	R0,SUM
Assembler directives		RETURN	
		END	START

Figure 2.18 Assembly language representation for the program in Figure 2.17.

Q. Draw the format of an instruction. Explain different fields in the format of an instruction.

GENERAL FORMAT OF A STATEMENT

- Most assembly languages require statements in a source program to be written in the form:

Label	*Operation*	*Operands*	*Comment*

1) Label is an optional name associated with the memory-address where the machine language instruction produced from the statement will be loaded.
2) Operation Field contains the OP-code mnemonic of the desired instruction or assembler.
3) Operand Field contains addressing information for accessing one or more operands, depending on the type of instruction.
4) Comment Field is used for documentation purposes to make program easier to understand.

Q. What is Assembler? List the functions of Assembler.

- Programs written in an assembly language are automatically translated into a sequence of machine instructions by the **Assembler**.
- **Assembler Program**
  → replaces all symbols denoting operations & addressing-modes with binary-codes used in machine instructions.
  → replaces all names and labels with their actual values.

→ assigns addresses to instructions & data blocks, starting at address given in ORIGIN directive
→ inserts constants that may be given in DATAWORD directives.
→ reserves memory-space as requested by RESERVE directives.

Q. What is Two Pass Assembler?

- **Two Pass Assembler** has 2 passes:

1) First Pass: Work out all the addresses of labels.

➢ As the assembler scans through a source-program, it keeps track of all names of numerical- values that correspond to them in a symbol-table.

2) Second Pass: Generate machine code, substituting values for the labels.

➢ When a name appears a second time in the source-program, it is replaced with its value from the table.

Q. Define the following:

a) Loader Program

- The assembler stores the object-program on a magnetic-disk. Loader program is used to load the object-program into the memory of the computer before it is executed.

b) Debugger Program

- **Debugger Program** is used to help the user find the programming errors.
- Debugger program enables the user
  → to stop execution of the object-program at some points of interest &
  → to examine the contents of various processor-registers and memory-location.

Q. With a neat diagram explain how a character is transferred from keyboard to processor.

- Consider the problem of moving a character-code from the keyboard to the processor (Figure: 2.19). For this transfer, buffer-register DATAIN & a status control flags(SIN) are used.
- When a key is pressed, the corresponding ASCII code is stored in a **DATAIN** register associated with the keyboard.
  - ➢ **SIN=1** ; When a character is typed in the keyboard. This informs the processor that a valid character is in DATAIN.
  - ➢ **SIN=0** ; When the character is transferred to the processor.
- An analogous process takes place when characters are transferred from the processor to the display. For this transfer, buffer-register DATAOUT & a status control flag SOUT are used.
  - ➢ **SOUT=1** ; When the display is ready to receive a character.
  - ➢ **SOUT=0** ; When the character is being transferred to DATAOUT.
- The buffer registers DATAIN and DATAOUT and the status flags SIN and SOUT are part of circuitry commonly known as a **device interface**.
-

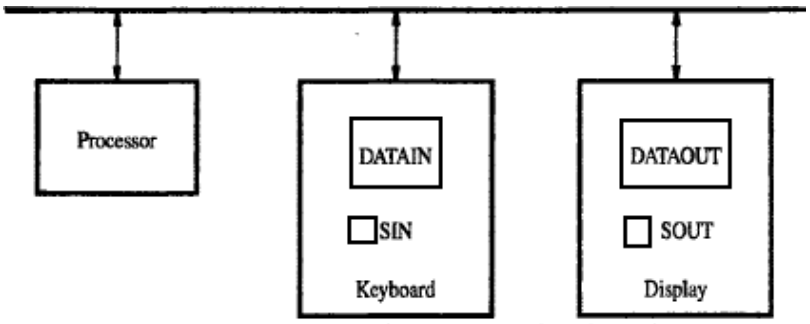


Figure 2.19 Bus connection for processor, keyboard, and display.

Q. Write a program that reads a line of characters and displays it.

• Some address values are used to refer to peripheral device buffer-registers such as DATAIN & DATAOUT. No special instructions are needed to access the contents of the registers; data can be transferred between these registers and the processor using instructions such as Move, Load or Store.

• For example, contents of the keyboard character buffer DATAIN can be transferred to register R1 in the processor by the instruction.

MoveByte DATAIN,R1

• The MoveByte operation code signifies that the operand size is a byte.

• The **Testbit** instruction tests the state of one bit in the destination, where the bit position to be tested is indicated by the first operand.

Program to read a line of characters and display it

	Move	#LOC,R0	Initialize pointer register R0 to point to the address of the first location in memory where the characters are to be stored.
READ	TestBit	#3,INSTATUS	Wait for a character to be entered
	Branch=0	READ	in the keyboard buffer DATAIN.
	MoveByte	DATAIN,(R0)	Transfer the character from DATAIN into the memory (this clears SIN to 0).
ECHO	TestBit	#3,OUTSTATUS	Wait for the display to become ready.
	Branch=0	ECHO	
	MoveByte	(R0),DATAOUT	Move the character just read to the display buffer register (this clears SOUT to 0).
	Compare	#CR,(R0)+	Check if the character just read is CR (carriage return). If it is not CR, then
	Branch≠0	READ	branch back and read another character. Also, increment the pointer to store the next character.

Figure 2.20 A program that reads a line of characters and displays it.

Q. What is Stack? Explain stack operations with neat diagram. OR
Q. Define Stack. Explain PUSH and POP operations on stack with neat sketches and examples.

• A **stack** is a special type of data structure where elements are inserted from one end and elements are deleted from the same end. This end is called the **top** of the stack (Figure: 2.14).

• The various operations performed on stack:

1) Insert: An element is inserted from top end. Insertion operation is called **push** operation.
2) Delete: An element is deleted from top end. Deletion operation is called **pop** operation.

• A processor-register is used to keep track of the address of the element of the stack that is at the top at any given time. This register is called the **Stack Pointer (SP)**.

• If we assume a byte-addressable memory with a 32-bit word length,

1) The push operation can be implemented as

Subtract #4, SP Move
NEWITEM, (SP)

2) The pop operation can be implemented as

Move (SP), ITEM
Add #4, SP

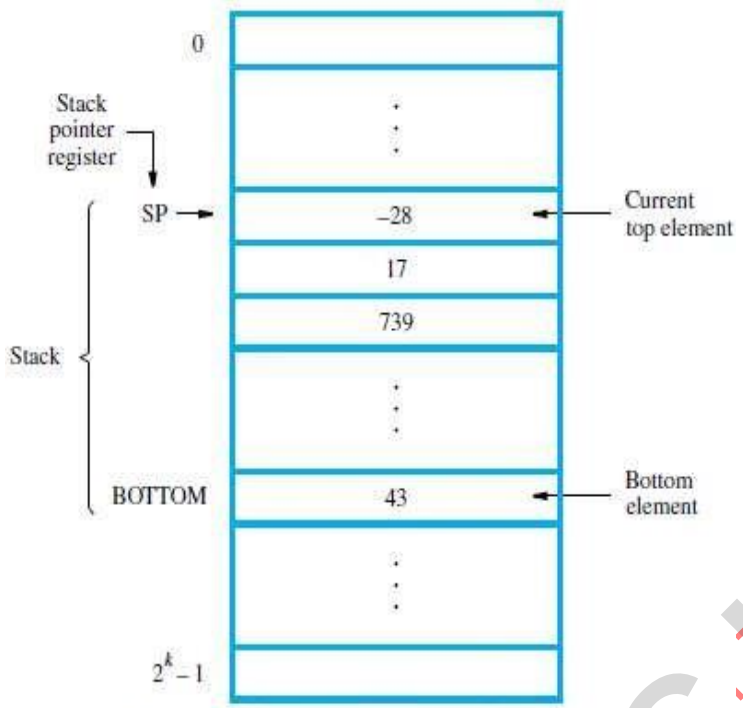


Figure 2.14 A stack of words in the memory.

Q. Write a program that checks for empty and full errors in pop and push operations. OR
Q. Write a program routine for a safe push and safe pop operations.

SAFEPOP	Compare Branch>0	#2000,SP EMPTYERROR	Check to see if the stack pointer contains an address value greater than 2000. If it does, the stack is empty. Branch to the routine EMPTYERROR for appropriate action.
	Move	(SP)+,ITEM	Otherwise, pop the top of the stack into memory location ITEM.

(a) Routine for a safe pop operation

SAFEPUSH	Compare Branch≤0	#1500,SP FULLERROR	Check to see if the stack pointer contains an address value equal to or less than 1500. If it does, the stack is full. Branch to the routine FULLERROR for appropriate action.
	Move	NEWITEM,–(SP)	Otherwise, push the element in memory location NEWITEM onto the stack.

(b) Routine for a safe push operation

Figure 2.23 Checking for empty and full errors in pop and push operations.

Q. Write a note on Reverse Polish Notation.

Reverse Polish Notation

- Infix Notation
  $A + B$
- Prefix or Polish Notation
  $+ A B$
- Postfix or Reverse Polish Notation (RPN)
  $A B +$

$A * B + C * D \xrightarrow{RPN} A B * C D * +$

(2) (4) * (3) (3) * +
(8) (3) (3) * +
(8) (9) +
17

Reverse Polish Notation

- Example

$(A + B) * [C * (D + E) + F]$

$(A B +) \quad (D E +) \quad C * F + *$

Reverse Polish notation (RPN) is a method for conveying mathematical expressions without the use of separators such as brackets and parentheses. In this notation, the operators follow their operands, hence removing the need for brackets to define evaluation priority.

Reverse Polish Notation is where the operator is written after its operands.
For example, AB+ is reverse Polish for A+B.

AB+C* is clearly (A+B)*C

That is if you evaluate A B + C * one operator at a time it goes:
A B + C * -> (A B +) * C -> (A+B)*C
or with numbers to make it clearer:
2 3 + 4 * -> 2+3 4 * -> 5 4 * -> 5*4 -> 20

Q. What is Queue? Explain the operations of Queue.

It is an ordered group of homogeneous items of elements.

- Data are stored in and retrieved from a queue on a FIFO basis.
- Difference between stack and queue?

1) One end of the stack is fixed while the other end rises and falls as data are pushed and popped.
2) In stack, a single pointer is needed to keep track of top of the stack at any given time.
   In queue, two pointers are needed to keep track of both the front and end for removal and insertion respectively.
3) Without further control, a queue would continuously move through the memory of a computer in the direction of higher addresses. One way to limit the queue to a fixed region in memory is to use a circular buffer.

Q. Compare Stack and Queue.

Stack	Queue
The stack is based on LIFO(Last In First Out) principle	The queue is based on FIFO(First In First Out) principle.
Insertion Operation is called Push Operation	Insertion Operation is called Enqueue Operation
Deletion Operation is called Pop Operation	Deletion Operation is called Dequeue Operation
Push and Pop Operation takes place from one end of the stack	Enqueue and Dequeue Operation takes place from a different end of the queue
The most accessible element is called Top and the least accessible is called the Bottom of the stack	The insertion end is called Rear End and the deletion end is called the Front End.
Simple Implementation	Complex implementation in comparison to stack
Only one pointer is used for performing operations	Two pointers are used to perform operations
Empty condition is checked using Top==-1	Empty condition is checked using Front==-1||Front==Rear+1
Full condition is checked using Top==Max-1	Full condition is checked using Rear==Max-1
There are no variants available for stack	There are three types of variants i.e circular queue, double-ended queue and priority queue
Can be considered as a vertical collection visual	Can be considered as a horizontal collection visual
Used to solve the recursive type problems	Used to solve the problem having sequential processing

Q. What is Subroutine? Explain subroutine linkage with neat diagram. Or

Q. Explain call and return instructions execution with neat diagram.

- A subtask consisting of a set of instructions which is executed many times is called a **Subroutine**.
- A Call instruction causes a branch to the subroutine (Figure: 2.16).
- At the end of the subroutine, a return instruction is executed
- Program resumes execution at the instruction immediately following the subroutine call
- The way in which a computer makes it possible to call and return from subroutines is referred to as its **Subroutine Linkage** method.
- The simplest subroutine linkage method is to save the return-address in a specific location, which may be a register dedicated to this function. Such a register is called the **Link Register**.
- When the subroutine completes its task, the Return instruction returns to the calling-program by branching indirectly through the link-register.
- The **Call Instruction** is a special branch instruction that performs the following operations:
  → Store the contents of PC into link-register.
  → Branch to the target-address specified by the instruction.
- The **Return Instruction** is a special branch instruction that performs the operation:
  → Branch to the address contained in the link-register.

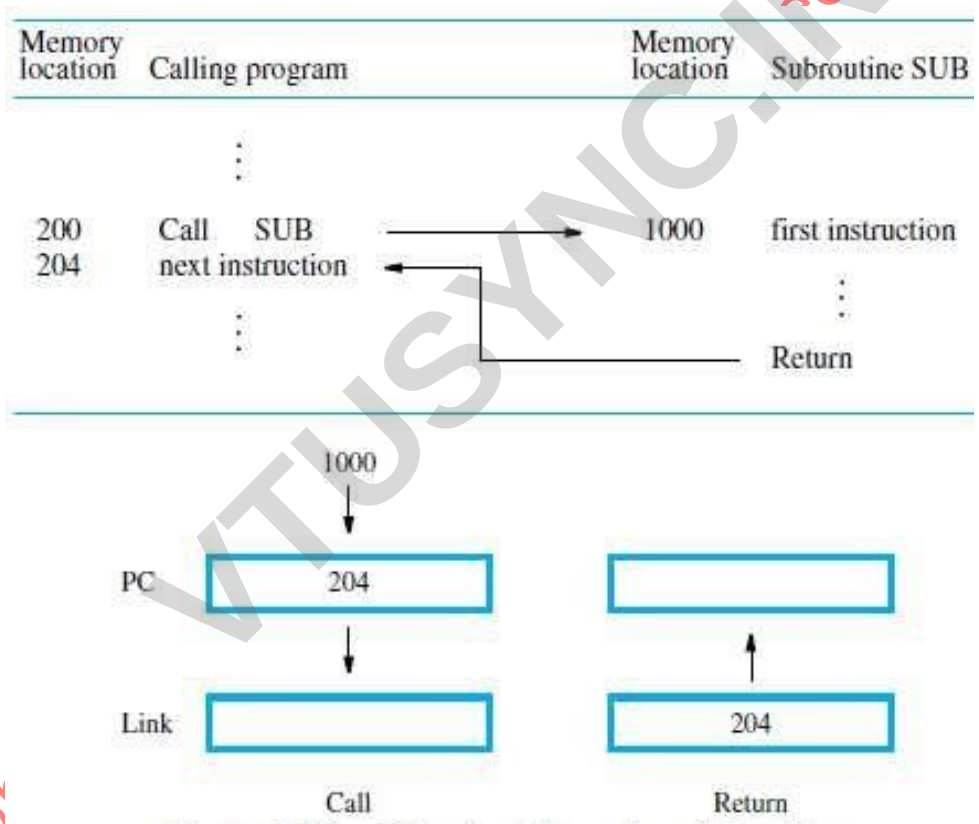


Figure 2.16 Subroutine linkage using a link register.

Q. What is subroutine nesting? Explain the role of processor stack in subroutine nesting.

- **Subroutine Nesting** means one subroutine calls another subroutine.
- In this case, the return-address of the second call is also stored in the link-register, destroying its previous contents.
- Hence, it is essential to save the contents of the link-register in some other location before calling another subroutine. Otherwise, the return-address of the first subroutine will be lost.
- Subroutine nesting can be carried out to any depth. Eventually, the last subroutine called completes its computations and returns to the subroutine that called it.
- The return-address needed for this first return is the last one generated in the nested call sequence. That is, return-addresses are generated and used in a LIFO order.
- This suggests that the return-addresses associated with subroutine calls should be pushed onto a stack. A particular register is designated as the SP(Stack Pointer) to be used in this operation.
- SP is used to point to the processor-stack.
- Call instruction pushes the contents of the PC onto the processor-stack.
  Return instruction pops the return-address from the processor-stack into the PC.

Q. What is parameter passing? Explain parameter passing with an example. OR Q. What is subroutine? With a psedudocode or program segment illustrate parameter passing using registers.

- A subtask consisting of a set of instructions which is executed many times is called a **Subroutine**.

PARAMETER PASSING

- The exchange of information between a calling-program and a subroutine is referred to as **Parameter Passing** (Figure: 2.25).
- The parameters may be placed in registers or in memory-location, where they can be accessed by the subroutine.
- Alternatively, parameters may be placed on the processor-stack used for saving the return-address.
- Following is a program for adding a list of numbers using subroutine with the parameters passed through registers.

Calling program

```
          Move      N,R1        R1 serves as a counter.
          Move      #NUM1,R2    R2 points to the list.
          Call      LISTADD     Call subroutine.
          Move      R0,SUM      Save result.
          :
```

Subroutine

```
LISTADD   Clear     R0          Initialize sum to 0.
LOOP      Add       (R2)+,R0    Add entry from list.
          Decrement R1
          Branch>0  LOOP
          Return                Return to calling program.
```

Figure 2.25 Program of Figure 2.16 written as a subroutine; parameters passed through registers.

Q. Explain the types of Logical Shift instructions with example for each.

- There are many applications that require the bits of an operand to be shifted right or left some specified number of bit positions.
- The details of how the shifts are performed depend on whether the operand is a signed number or some more general binary-coded information.
- For general operands, we use a logical shift.

For a number, we use an arithmetic shift, which preserves the sign of the number.

LOGICAL SHIFTS

- Two logical shift instructions are
  1) Shifting left (LShiftL) &
  2) Shifting right (LShiftR).
- These instructions shift an operand over a number of bit positions specified in a count operand contained in the instruction.

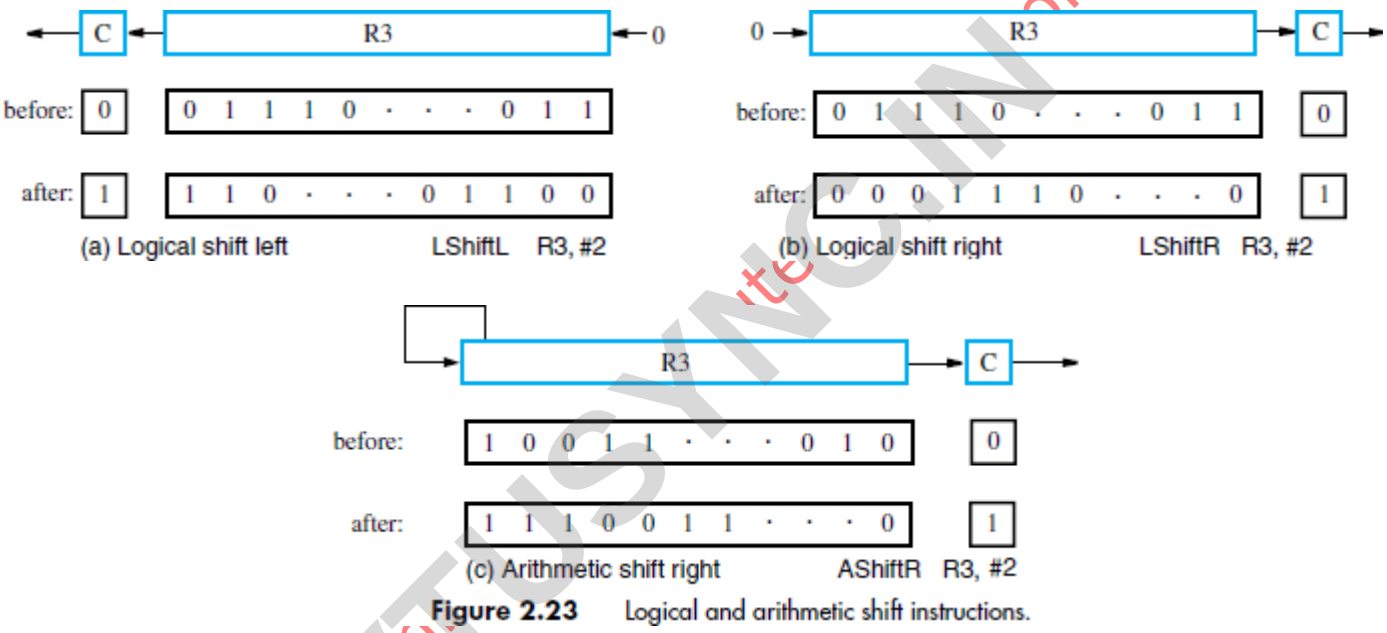


Figure 2.23 Logical and arithmetic shift instructions.

Q. Write a program that packs two BCD digits using logical shift instruction.

Move	#LOC,R0	R0 points to data.
MoveByte	(R0)+,R1	Load first byte into R1.
LShiftL	#4,R1	Shift left by 4 bit positions.
MoveByte	(R0),R2	Load second byte into R2.
And	#$F,R2	Eliminate high-order bits.
Or	R1,R2	Concatenate the BCD digits.
MoveBvte	R2.PACKED	Store the result.

Figure 2.31 A routine that packs two BCD digits.

Q. Explain the types of rotate instructions with example for each.

- In shift operations, the bits shifted out of the operand are lost, except for the last bit shifted out which is retained in the Carry-flag C.
- To preserve all bits, a set of rotate instructions can be used.
- They move the bits that are shifted out of one end of the operand back into the other end.
- Two versions of both the left and right rotate instructions are usually

• provided. In one version, the bits of the operand is simply rotated.
In the other version, the rotation includes the C flag.

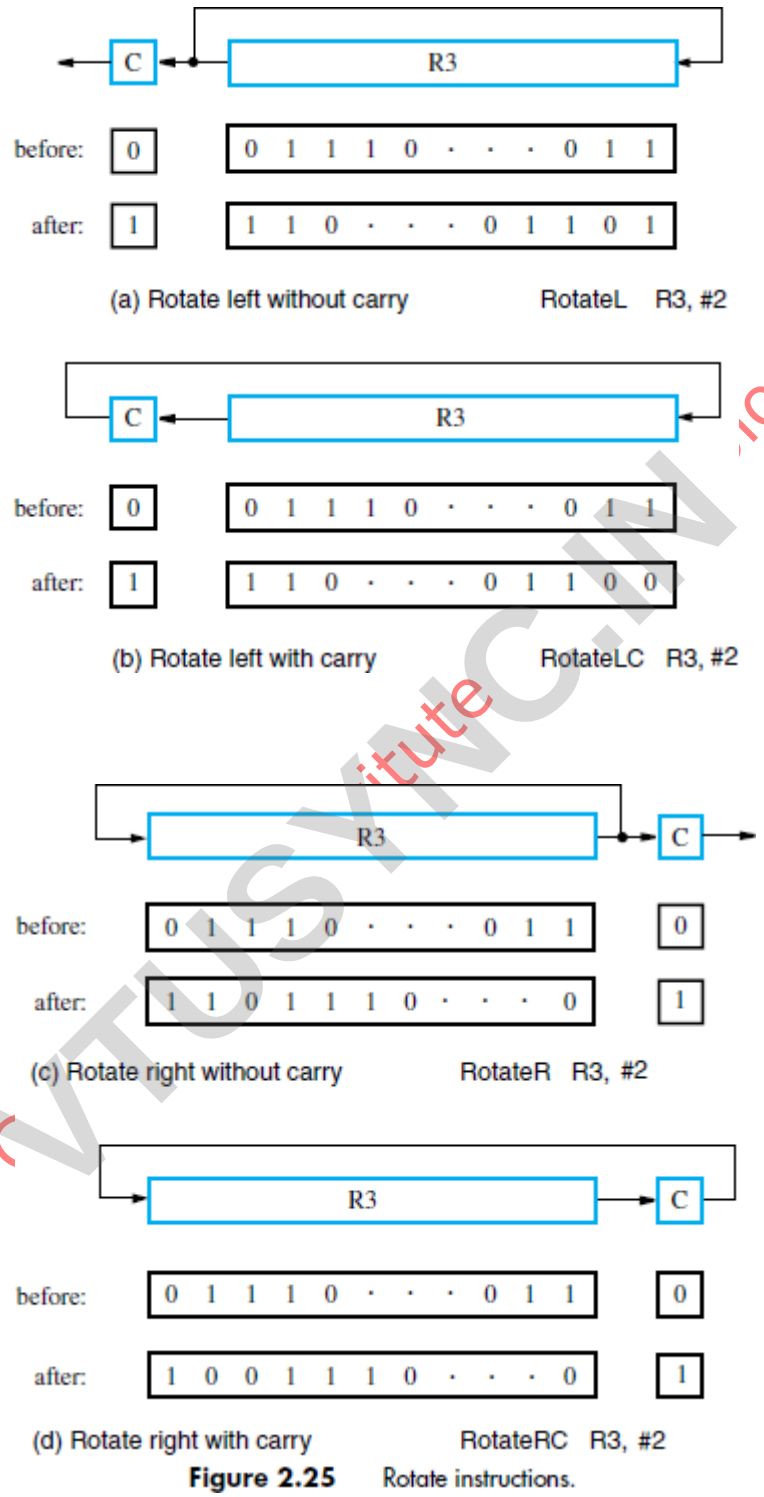


Figure 2.25 Rotate instructions.

Problem 1:

Write a program that can evaluate the expression A*B+C*D In a single-accumulator processor. Assume that the processor has Load, Store, Multiply, and Add instructions and that all values fit in the accumulator.

Solution:

A program for the expression is:

Load A
Multiply B Store RESULT Load C
Multiply D Add RESULT
Store RESULT

Problem 2:

Registers R1 and R2 of a computer contains the decimal values 1200 and 4600. What is the effective- address of the memory operand in each of the following instructions?

(a) Load 20(R1), R5
(b) Move #3000,R5
(c) Store R5,30(R1,R2)
(d) Add -(R2),R5
(e) Subtract (R1)+,R5

Solution:

(a) EA = [R1]+Offset=1200+20 = 1220
(b) EA = 3000
(c) EA = [R1]+[R2]+Offset = 1200+4600+30=5830 (d) EA = [R2]-1 = 4599
(e) EA = [R1] = 1200

Problem 3:

Registers R1 and R2 of a computer contains the decimal values 2900 and 3300. What is the effective-address of the memory operand in each of the following instructions?

(a) Load R1,55(R2)
(b) Move #2000,R7
(c) Store 95(R1,R2),R5
(d) Add (R1)+,R5
(e) Subtract-(R2),R5

Solution:

a) Load R1,55(R2) ; This is indexed addressing mode. So EA =55+R2=55+3300=3355.
b) Move #2000,R7 ; This is an immediate addressing mode. So, EA =2000
c) Store 95(R1,R2),R5 ; This is a variation of indexed addressing mode, in which contents of 2 registers are added with the offset or index to generate EA. So, 95+R1+R2=95+2900+3300=6255.
d) Add (R1)+,R5 ; This is Autoincrement mode. Contents of R1 are the EA so, 2900 is the EA.
e) Subtract -(R2),R5 ; This is Auto decrement mode. Here, R2 is subtracted by 4 bytes (assuming 32-bt processor) to generate the EA, so, EA= 3300-4=3296.

Problem 4:

Given a binary pattern in some memory-location, is it possible to tell whether this pattern represents a machine instruction or a number?

Solution:

No; any binary pattern can be interpreted as a number or as an instruction.

Problem 5:

Both of the following statements cause the value 300 to be stored in location 1000, but at different times.

ORIGIN 1000
DATAWORD 300
And
Move #300, 1000
Explain the difference.

Solution:

The assembler directives ORIGIN and DATAWORD cause the object program memory image constructed by the assembler to indicate that 300 is to be placed at memory word location 1000 at the time the program is loaded into memory prior to execution.

The Move instruction places 300 into memory word location 1000 when the instruction is executed as part of a program.

Problem 6:

Register R5 is used in a program to point to the top of a stack. Write a sequence of instructions using the Index, Autoincrement, and Autodecrement addressing modes to perform each of the following tasks:

(a) Pop the top two items off the stack, add them, and then push the result onto the stack.

Solution:

(a) Move (R5)+,R0
Add (R5)+,R0
Move R0,-(R5)

Problem 7:

Consider the following possibilities for saving the return address of a subroutine:

(a) In the processor register.

(b) In a memory-location associated with the call, so that a different location is used when the subroutine is called from different places

(c) On a stack. Which of these possibilities supports subroutine nesting and which supports subroutine recursion(that is, a subroutine that calls itself)?

Solution:

(a) Neither nesting nor recursion is supported.

(b) Nesting is supported, because different Call instructions will save the return address at different memory-locations. Recursion is not supported.

Problem 8:

Consider a set of numbers stored in memory starting at address TABLE. Total numbers are N and this value is stored at location LOCN. Develop an ALP using auto increment addressing mode, to compute the sum of all numbers and store the result at memory address ResultL. Write appropriate comments.

```
        Move  LOCN, R1   ; Count N  is stored into R1 register.
        Move #TABLE, R2  ; Starting address of the numbers stored is moved to R2
                           register.
        Clear R3         ;  Register R3 is filled with zero.
Label:  Add (R2)+,R3     ; perform addition operation.
        Decrement R1     ; decrement count value.
        Branch<0, Label
        Store R1, RESULTL
```